Lecture 1 — Training at the Championship Level

> National Championship Prep · Prerequisites: Year Curriculum, Regional track (especially regional-01 *Contest anatomy* and week-35 *Contest strategy*) · Estimated time: 90 min lecture + ongoing weekly routine

1. Overview

Getting to a regional final is mostly about **knowing the standard toolkit** and **not making mistakes on the easy half of the set**. Getting a good result at a national championship (ICPC North America Championship, a national olympiad final, a Codeforces Div. 1 round) is different in three ways:

1. **The easy half is solved by everyone.** Rank is decided by the 3–5 problems in the medium-hard band: problems that need a known advanced technique *plus* a non-obvious observation.
2. **Implementation volume goes up.** A heavy-light decomposition with a lazy segment tree, a min-cost flow model, a suffix automaton — each is 60–150 lines that must work the first time.
3. **The difference between teams is the system, not the talent.** At this level all teams are strong; the teams that win are the ones that practise deliberately, upsolve everything, use a battle-tested notebook and run the computer like a shared resource.

This lecture sets up the training system the rest of the track assumes. Every later lecture (advanced data structures in Lecture 2, DP optimizations in Lecture 3, tree decompositions in Lecture 4, flows in Lecture 5, then strings, math, geometry and graphs, ending with the mock championship in Lecture 10) feeds into the same loop: **learn → implement into the notebook → apply in a virtual contest → upsolve → review**.

2. What "championship level" means

A rough calibration (it varies by year and country):

Signal	Regional finalist	National championship contender
Individual Codeforces rating	~1700–2000	~2100–2500+
Div. 1 round	solve A, sometimes B	solve A–B reliably, C regularly

Signal	Regional finalist	National championship contender
ICPC regional set	solve 6–8	solve 9+ of 11–13
Known techniques	segment tree, Dijkstra, basic DP, max-flow	everything in this track, from memory or notebook

The table is not a gate; it is a way to set measurable goals. Pick goals that are **process goals** ("upsolve every problem rated ≤ my rating + 300 within a week") rather than outcome goals ("reach 2400"), because process goals are fully under your control.

The skills that separate teams

- **Recognition speed.** Seeing "this is a min-cost flow" or "the cost satisfies the quadrangle inequality" within minutes. Built by volume and by reviewing *why* you missed recognitions.
- **Implementation reliability.** Writing 120 lines with no bug, or finding the bug in 5 minutes. Built by implementing each notebook entry from scratch several times and by stress testing.
- **Proof habit.** At this level many wrong greedy ideas pass samples. A team that writes a 3-line proof sketch (exchange argument, monotonicity of the optimum, invariant) before coding wastes far fewer penalty minutes.
- **Team coordination.** Three people, one computer, five hours. The computer is the bottleneck: nobody should be thinking at the keyboard.

3. The weekly training loop

A sustainable schedule for a team training for a national championship over ~10–15 weeks:

Day	Activity	Time
1	Lecture + implement the lecture's templates from scratch	2–3 h
2–3	Individual topic practice (5–8 problems from the lecture list)	2 h/day
4	**Team virtual contest** (full 5 h, one computer)	5 h
5	Team post-mortem + start upsolving	1–2 h
6–7	Individual contest (Codeforces / AtCoder) + upsolving	2–4 h

3.1 Virtual contests

A virtual contest is a past contest done under real conditions. Rules that make it useful:

- **Use real sets of the right level.** Past NAC sets, other national championships, strong regionals (e.g. NWERC, CERC, Asia regionals), and World Finals sets near the end of training. Kattis and the Codeforces Gym host many of them.

- **Simulate everything.** One computer, printed notebook only, 5 hours, no internet except the judge, scoreboard frozen for the last hour if your simulator supports it.
- **Log events.** One person keeps a time-stamped log: when each problem was read, who took it, when it was submitted, verdicts. The log is the raw material of the post-mortem.

3.2 Upsolving

Upsolving is solving, after the contest, the problems you did not solve during it. It is where most learning happens.

Protocol:

1. For each unsolved problem, spend **at least 30–60 minutes** thinking on your own before reading the editorial — the struggle is what makes the idea stick.
2. Read only the **first hint** of the editorial (the first paragraph), then try again.
3. After reading the full solution, **write it yourself** from an empty file. Do not copy code.
4. Record in a shared spreadsheet: problem, technique, *the observation you missed*, and how you could have found it. The third column is the most valuable data your team owns.
5. If a technique was missing from your notebook, add it (see §5).

A good rule: **upsolve every problem solved by at least 1/4 of the teams that finished above you.** Problems solved by nobody can wait.

3.3 Post-mortem

Within 24 hours of a team contest, spend 45 minutes on these questions:

- **Time lost.** Where did the time go? Look for gaps in the log: a problem that took 90 minutes from reading to AC, a 30-minute debugging session at the computer, a problem nobody read until hour 3.
- **Penalty.** Every WA: was it a bug a stress test would have caught? An overflow? A misread statement?
- **Missed problems.** Which unsolved problem was solvable by us today? What stopped us?
- **One change.** Agree on *one* concrete process change for the next contest. Changing five things at once teaches you nothing.

4. Team roles and computer management

With three people and one machine, the main rule is: **the keyboard is only for typing code that has already been designed.**

Roles at the start

A common opening (adapt to your team):

- **Coder A** sets up the environment (template, compile script, `stress.sh`) in the first 2–3 minutes, then codes the first easy problem.
- **Reader B** reads problems from the front of the set, **Reader C** from the back. Each writes a one-line summary and estimated difficulty on a shared sheet.
- After ~20 minutes everyone has a picture of the set; the scoreboard confirms which problems are easy.

During the contest

- **Pair on hard problems, solo on easy ones.** Two people agreeing on an approach (and a proof sketch) before coding prevents most wasted hours.
- **Print and debug on paper.** If a submission fails and the bug is not obvious in 2 minutes, print the code and give the keyboard to the next person.
- **Queue discipline.** Keep an explicit queue: "after A finishes problem F, B codes problem K (est. 25 min)". Write it on paper.
- **Specialization, not exclusivity.** It helps if one person "owns" geometry and another "owns" strings — they keep those notebook sections and handle those problems first. But everyone must be able to code a segment tree or Dijkstra.
- **Last hour.** Stop opening new problems with less than ~45 minutes left unless the solution is clear; focus the whole team on the one or two problems most likely to pass.

Stress testing at the computer

Week 35 introduced stress testing; at this level it is mandatory for any WA on a problem with a small-input brute force. Keep a ready script so that it costs 3 minutes, not 15 (see §6). The person writing the brute force should **not** be the person who wrote the solution, so that the same misunderstanding does not appear in both.

5. The team notebook (TRD)

ICPC allows a printed team reference document (at the World Finals this is limited to 25 single-sided pages; regional and national contests usually have similar rules — always check the current rules for your contest). The notebook is a training tool as much as a contest tool.

What goes in

Prioritize code that is **long, error-prone, and needed occasionally**:

- Data structures: lazy segment tree, persistent segment tree, implicit treap, Li Chao tree (Lectures 2–3).
- Trees: heavy-light decomposition, centroid decomposition (Lecture 4).
- Flows: Dinic, min-cost flow, Hopcroft–Karp, lower bounds (Lecture 5).
- Strings: suffix array + LCP, suffix automaton, Aho–Corasick (Lecture 6, week 29).
- Math: modular arithmetic, CRT, NTT, Gaussian elimination, XOR basis, Miller–Rabin/Pollard rho (Lecture 7).
- Geometry: point/line primitives, convex hull, half-plane intersection, rotating calipers (Lecture 8).
- Graphs: SCC, 2-SAT, bridges/articulation points, Euler tour (Lecture 9).

What does **not** go in: things you can write in 5 lines from memory (BFS, binary search, DSU), or long explanatory prose.

Rules for notebook code

1. **Every entry is tested.** Each entry has been submitted to at least one judge problem and stress-tested against a brute force. Record the problem ID in a comment at the top.
2. **Uniform interface.** Same naming, 0- vs 1-indexing documented in the first line, `ll` everywhere sums can overflow.
3. **Compact but readable.** Dense formatting is fine; clever macros are not — at 4 hours into a contest you must be able to read it.
4. **State complexity and preconditions.** E.g. "MCMF: no negative cycles; O(F·m log n)".
5. **Typing practice.** Each team member should have typed each entry at least twice. The notebook is a memory aid, not a substitute for understanding.

Public notebooks such as KACTL (KTH) are excellent models for style and content. Use them for inspiration, but build and test your own — you must know every line.

6. Reference implementation: the contest-day kit

6.1 Template

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using pii = pair<int, int>;
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
#ifdef LOCAL
#define dbg(...) cerr << "[" << #__VA_ARGS__ << "] = ", dbgPrint(__VA_ARGS__)
template <class T> void dbgPrint(const T& x) { cerr << x << endl; }
template <class T, class... R> void dbgPrint(const T& x, const R&... r) { cerr << x << ",
"; dbgPrint(r...); }
```

```cpp
#else
#define dbg(...) ((void)0)
#endif

void solve() {
    int n; cin >> n;
    vector<ll> a(n);
    for (auto& x : a) cin >> x;
    dbg(n, sz(a));
    cout << *max_element(all(a)) << "\n";
}

int main() {
    cin.tie(nullptr)->sync_with_stdio(false);
    int t = 1;
    cin >> t;
    while (t--) solve();
}
```

Compile locally with `-DLOCAL` to enable `dbg`; the judge build silently drops it. Also compile locally with `-fsanitize=address,undefined -D_GLIBCXX_DEBUG` when chasing a runtime error: out-of-bounds `vector` accesses then crash with a message instead of producing garbage.

6.2 Random generator

```cpp
#include <bits/stdc++.h>
using namespace std;
// usage: ./gen <seed>   -- prints one small random test
int main(int argc, char* argv[]) {
    mt19937 rng(atoi(argv[1]));
    auto rnd = [&](int lo, int hi) { return uniform_int_distribution<int>(lo, hi)(rng);
};
    int n = rnd(1, 8);                      // SMALL: the failing test must be readable
    cout << n << "\n";
    for (int i = 0; i < n; i++) cout << rnd(-10, 10) << " \n"[i == n - 1];
}
```

Seeding by the command-line argument makes every failing test reproducible.

6.3 Stress script

```bash
#!/usr/bin/env bash
# usage: ./stress.sh [iterations]   (expects sol.cpp, brute.cpp, gen.cpp in this folder)
set -e
for f in sol brute gen; do g++ -std=c++17 -O2 -Wall -o "$f" "$f.cpp"; done
for ((i = 1; i <= ${1:-1000}; i++)); do
    ./gen "$i" > in.txt
    ./sol < in.txt > out_sol.txt
    ./brute < in.txt > out_brute.txt
    if ! cmp -s out_sol.txt out_brute.txt; then
```

```
        echo "MISMATCH on seed $i"; echo "--- input"; cat in.txt
        echo "--- sol";   cat out_sol.txt; echo "--- brute"; cat out_brute.txt
        exit 1
    fi
done
echo "all ${1:-1000} tests passed"
```

Demo: with `sol.cpp` = Kadane's algorithm with the classic bug of resetting the running sum to 0 (so an all-negative array answers 0), and `brute.cpp` = the O(n²) maximum non-empty subarray, the script reports within a few seeds:

```
MISMATCH on seed 7
--- input
1
-6
--- sol
0
--- brute
-6
```

When the outputs are not unique (any valid answer accepted), replace `cmp` with a small checker program that validates `out_sol.txt` against the input.

6.4 The upsolve log

A shared spreadsheet with columns:

Date	Contest	Problem	Technique	Key observation missed	Notebook entry?	Upsolved by

Review it monthly and look for repeated rows: if "didn't see the binary search on the answer" appears four times, that is your next training topic.

7. Common mistakes

- **Training only individually.** Individual rating helps, but team contests train the coordination skills that decide championships. Do at least one full team contest per week in the final two months.
- **Reading editorials too early.** Ten minutes of struggle followed by the editorial gives the illusion of learning. Use the hint-first protocol.
- **Untested notebook code.** A template that fails on edge cases is worse than no template, because you trust it. Every entry gets a judge AC and a stress test.
- **Thinking at the keyboard.** If the coder stops typing for more than a minute, they should hand over the machine.
- **Skipping the post-mortem.** Without it, the same 40-minute mistake repeats every week.

- **Not stress testing after a WA.** Re-reading code for 30 minutes is usually slower than a 3-minute stress test with a ready script.
- **Ignoring easy problems in the set.** At a national championship, penalty time on the easy problems decides ties among teams with the same count. Solve them fast and cleanly.
- **Overfitting to one problem source.** Different contests have different styles; rotate between Codeforces, AtCoder, Kattis/ICPC sets and olympiad archives.

8. Practice problems

This week's practice is about the *system*, not a single technique:

1. **Set up the kit.** Type the template, generator and stress script from memory; run the Kadane demo above. Then use it on CSES 1643 — Maximum Subarray Sum.
2. **Stress-test an off-by-one-prone problem.** CSES 1085 — Array Division (binary search on the answer): write a brute force over all splits for n ≤ 8 and stress test before submitting.
3. **Notebook seed.** Implement, judge and stress test a lazy segment tree on CSES 1735 — Range Updates and Sums. This becomes your first notebook page.
4. **Team virtual contest.** Pick a past ICPC national championship or strong regional set from Kattis problem sources (e.g. an ICPC North America Championship or NWERC set) or the Codeforces Gym. Run it under full conditions, keep the event log, and do the post-mortem.
5. **Individual contests.** One live or virtual Codeforces Div. 1 or Div. 2 round and one AtCoder Regular Contest this week; upsolve per the protocol.
6. **Recognition drill.** Take 10 problems from the CSES Advanced Techniques section and, for each, spend 5 minutes writing only the technique you would use. Check against solutions; log misses.

9. Further reading

- KACTL — KTH Algorithm Competition Template Library: https://github.com/kth-competitive-programming/kactl (model team notebook).
- cp-algorithms: https://cp-algorithms.com — reference for nearly every technique in this track.
- Antti Laaksonen, *Competitive Programmer's Handbook* (free PDF from https://cses.fi/book/) and *Guide to Competitive Programming* (Springer).
- USACO Guide, Advanced section: https://usaco.guide/adv/ — structured problem lists by technique.
- ICPC rules and past problem archives: https://icpc.global and the ICPC World Finals problem archive.
- Previous lectures: week-35 (*Contest strategy, debugging & stress testing*), regional-01 (*Anatomy of a regional*), regional-10 (*Mock regional*). Next: Lecture 2 (*Advanced data*

structures), and Lecture 10 (*Mock national championship*) where this system is put to the test.